A Gentle Introduction to Prologos: Types, Specs, and Sessions

For Programmers Who Ship Code — A Spiral Curriculum

Prologos Project

2026-02-24

Contents

1 Who This Is For **3**

2 Spiral 1: Just Write Functions **3**
2.1 1.1 Your First Prologos Function 3
2.2 1.2 Pattern Matching 3
2.3 1.3 Collections Work Like You'd Expect 4

3 Spiral 2: Adding Type Signatures with `spec` **4**
3.1 2.1 Why Bother? 4
3.2 2.2 Level 1: Simple Types 4
3.3 2.3 Level 2: Generics (Polymorphism) 5
3.4 2.4 Level 3: Constrained Generics 5

4 Spiral 3: `spec` as Self-Documenting Metadata **5**
4.1 3.1 Beyond Types: Specs as Rich Descriptions 5
4.2 3.2 Why This Matters 6
4.3 3.3 The Graduation Path 6

5 Spiral 4: Higher-Kinded Types (Without the Jargon) **8**
5.1 4.1 The Problem: Writing Generic Code Over Containers . . 8
5.2 4.2 Prologos: Traits Over Containers 8
5.3 4.3 You Already Know This Pattern 8

6 Spiral 5: `functor` — Naming Complex Types **9**
6.1 5.1 The Problem: Type Signatures Get Long 9
6.2 5.2 The Solution: Name It 9
6.3 5.3 Progressive Metadata on `functor` 10
6.4 5.4 The Key Insight 10

7 Spiral 6: Channels and Protocols (Session Types) **10**
7.1 6.1 The Problem: Communication Goes Wrong 10
7.2 6.2 Session Types: Protocols as Types 10
7.3 6.3 Branching Protocols: Choice and Offer 11
7.4 6.4 Dependent Sessions: Protocols That Depend on Values . . 11

8 Spiral 7: Capabilities and the Eventual Send **12**
8.1 7.1 A Quick Grounding: What Are Capabilities? 12
8.2 7.2 Channels as Capabilities 12
8.3 7.3 Linear Types Prevent Capability Duplication 13
8.4 7.4 The Eventual Send 13
8.5 7.5 How This Compares to OCapN 14

9 Spiral 8: Dependent Types (The Deeper Layer) **15**
9.1 8.1 You've Already Used Them 15
9.2 8.2 The Full Progressive Complexity Ladder 15
9.3 8.3 `??` : The "What Goes Here?" Hole 16

10 Spiral 9: The Big Picture — `spec`, `functor`, `session` **17**
10.1 9.1 The Keyword Symmetry 17
10.2 9.2 They All Compose 17

11 Where To Go From Here **18**
11.1 The Progressive Complexity Promise 18

12 Further Reading **18**
12.1 Prologos Design Documents 18
12.2 The Codebase 19
12.3 Capability-Based Security 19

> "The language is actually designed to be minimal in ceremony and syntax, and look friendly and inviting to any programmer generally familiar with functional programming; although there are layers of depth to unlock further expressiveness, if they choose to go there."

1 Who This Is For

You write JavaScript or TypeScript. You've shipped production services. You might know about capability-based security from the OCapN community, or about distributed systems from building real infrastructure. You've heard terms like "dependent types" and "session types" but they sound academic.

This tutorial shows you that Prologos's type system is *not* academic. It's a pragmatic tool for catching bugs you've been catching with tests, conventions, and code review. You can use as little or as much of it as you want — the layers are opt-in.

Prerequisites: Familiarity with a typed language (TypeScript, Java, Rust) and comfort with first-class functions (`map`, `filter`, `reduce`).

What you won't need: Category theory, proof assistants, Greek letters, or a PhD.

2 Spiral 1: Just Write Functions

2.1 1.1 Your First Prologos Function

```
defn greet [name]
  [string-append "Hello, " name]
```

That's it. No type annotations. Prologos infers that `name` is a `String` and the return value is a `String`. If you've used TypeScript with `noImplicitAny` off, this feels familiar — write code, let the system figure out the types.

Key syntax note: `[]` is for function application. `defn greet [name]` defines a function `greet` that takes `name`. `[string-append "Hello, " name]` calls `string-append` with two arguments.

2.2 1.2 Pattern Matching

```
defn factorial
  | [0] -> 1
  | [n] -> [* n [factorial [- n 1]]]
```

The `|` arms define cases directly on the function — no separate `match` expression needed. Brackets `[...]` are for function calls. This reads naturally: "factorial of 0 is 1; factorial of n is n times factorial of (n - 1)."

2.3 1.3 Collections Work Like You'd Expect

```
;; List literal
def xs '[1 2 3 4 5]

;; PVec (persistent vector -- like Clojure's vectors)
def ys @[10 20 30]

;; Map (persistent hash map)
def user {:name "Alice" :age 30}

;; Set (persistent hash set)
def primes #{2 3 5 7}

;; Generic operations work on ALL of them
map inc xs                      ;; => '[2 3 4 5 6]
map inc ys                      ;; => @[11 21 31]
filter (fn [x] [> x 3]) xs     ;; => '[4 5]
reduce + 0 xs                   ;; => 15
into @[] xs                     ;; => @[1 2 3 4 5]  (List -> PVec)
```

`map`, `filter`, `reduce` work generically on Lists, PVecs, Sets — any collection type. If you've used Clojure's seq abstraction or Rust's iterators, this is the same idea: one set of function names, all collection types, type-preserving.

3 Spiral 2: Adding Type Signatures with `spec`

3.1 2.1 Why Bother?

Type inference is great for small functions. But as code grows, explicit type signatures become *documentation that the compiler checks*. TypeScript developers know this instinct — you *could* let TS infer everything, but `: number` on a function parameter prevents a whole class of bugs.

In Prologos, type signatures live in `spec` declarations — separate from the function body. Think of `spec` as TypeScript's `interface` for a single function:

3.2 2.2 Level 1: Simple Types

```
spec add : Int -> Int -> Int
```

```
defn add [x y] [+ x y]
```

Read this as: "`add` takes an `Int` and an `Int` and returns an `Int`." Arrow (`->`) separates arguments from return type.

This is the equivalent of TypeScript's:

```
function add(x: number, y: number): number { ... }
```

3.3 2.3 Level 2: Generics (Polymorphism)

```
spec id : A -> A
defn id [x] x
```

`A` is a type variable — just like TypeScript's `<T>`. Prologos infers that `A` is a type parameter because it appears free in the signature (starts with a capital letter, not a known type). No need to write the equivalent of `<A>`.

TypeScript equivalent:

```
function id<A>(x: A): A { return x; }
```

3.4 2.4 Level 3: Constrained Generics

What if your generic function needs the type to support certain operations?

```
spec sort : [List A] -> [List A]
  :where (Ord A)
defn sort [xs] ...
```

`:where (Ord A)` means "`A` must support ordering." This is like TypeScript's `<A extends Comparable<A>>` but more flexible — traits (Prologos's version of interfaces) are resolved automatically.

The user writes `sort '[3 1 2]` — the compiler sees `A = Int`, finds `Int`'s `Ord` instance, and passes it in automatically. Zero ceremony at the call site.

4 Spiral 3: `spec` as Self-Documenting Metadata

4.1 3.1 Beyond Types: Specs as Rich Descriptions

Here's where Prologos diverges from TypeScript. A `spec` isn't *just* a type signature — it's a structured metadata record. You can attach documentation, examples, and properties *right next to the type*:

```
spec sort : [List A] -> [List A]
  :where (Ord A)
  :doc "Sort a list in ascending order using merge sort"
  :examples
    - [sort '[3 1 2]] => '[1 2 3]
    - [sort '[]] => '[]
  :properties
    - :name "preserves-length"
      :holds [eq? [length [sort xs]] [length xs]]
    - :name "idempotent"
      :holds [eq? [sort [sort xs]] [sort xs]]
```

Let's break this down:

Key	What it does	Analogy
`:doc`	Human-readable description	JSDoc `@description`
`:examples`	Concrete input/output pairs	Jest `test("should...")`
`:properties`	Universal invariants (for all valid inputs)	QuickCheck / fast-check property
`:where`	Trait constraints (interfaces the type must have)	TS `extends` constraint

4.2 3.2 Why This Matters

In JavaScript, these live in *four different places*:

1. Types: TypeScript `.d.ts` files or inline annotations
2. Docs: JSDoc comments (often stale)
3. Examples: Unit tests in a separate `*.test.js` file
4. Properties: Property-based tests in a third file (if you have them at all)

In Prologos, they're *all part of the spec*. The compiler reads them. The tooling reads them. They're always in sync because they live next to the function they describe.

4.3 3.3 The Graduation Path

This is the key design philosophy: **you opt in to more precision**. Each level adds information without changing the previous levels:

```
;; Level 0: No types at all. Inference does the work.
defn add [x y] [+ x y]

;; Level 1: Add a type signature.
spec add : Int -> Int -> Int
defn add [x y] [+ x y]

;; Level 2: Add documentation.
spec add : Int -> Int -> Int
  :doc "Integer addition"
defn add [x y] [+ x y]

;; Level 3: Add examples.
spec add : Int -> Int -> Int
  :doc "Integer addition"
  :examples
    - [add 2 3] => 5
defn add [x y] [+ x y]

;; Level 4: Add properties.
spec add : Int -> Int -> Int
  :doc "Integer addition"
  :examples
    - [add 2 3] => 5
  :properties
    - :name "commutative"
      :holds [eq? [add x y] [add y x]]
    - :name "identity"
      :holds [eq? [add x 0] x]
defn add [x y] [+ x y]
```

At no point do you need to understand dependent types, Pi types, or category theory. You're just adding metadata that the compiler checks.

The implementation doesn't change. Only the specification becomes richer.

5 Spiral 4: Higher-Kinded Types (Without the Jargon)

5.1 4.1 The Problem: Writing Generic Code Over Containers

You know `map` works on arrays in JavaScript. But what if you want to write a function that works on *any container* — arrays, Sets, Maps, async iterables — without knowing which one?

In TypeScript, this is awkward. You'd need overloads or a complex generic:

```
// TypeScript: you end up with overloads
function fmap<A, B>(f: (a: A) => B, xs: Array<A>): Array<B>;
function fmap<A, B>(f: (a: A) => B, xs: Set<A>): Set<B>;
// ... one per container type
```

5.2 4.2 Prologos: Traits Over Containers

In Prologos, the `Functor` trait captures "any container you can map over":

```
spec fmap : [A -> B] -> [F A] -> [F B]
  :where (Functor F)
  :doc "Apply a function inside any container"
```

Read `F` as "some container type." `F A` means "a container of `A` values." The `:where (Functor F)` constraint says "`F` must be a type that supports mapping."

Usage — zero ceremony:

```
fmap inc '[1 2 3]            ;; => '[2 3 4]
fmap inc @[1 2 3]           ;; => @[2 3 4]
fmap to-string '[1 2]       ;; => '["1" "2"]
```

The compiler sees `F = List` or `F = PVec`, finds the right `Functor` instance, and dispatches automatically. This is like Rust's trait system or Haskell's type classes, but with less syntax.

5.3 4.3 You Already Know This Pattern

If you've used JavaScript's `Symbol.iterator` protocol, you know this idea:

```
// JS: any object with [Symbol.iterator] is iterable
for (const x of anyIterable) { ... }
```

Prologos traits are the typed version of protocols: any type that implements `Seqable` (the "can produce a sequence" trait) works with `map`, `filter`, `reduce`, etc. The compiler *proves* the implementation exists, rather than discovering at runtime that it doesn't.

6 Spiral 5: `functor` — Naming Complex Types

6.1 5.1 The Problem: Type Signatures Get Long

As your code becomes more generic, type signatures grow:

```
;; This is getting hard to read...
spec compose-xf : <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
                  -> <(S :0 Type) -> [S -> C -> S] -> S -> B -> S>
                  -> <(S :0 Type) -> [S -> C -> S] -> S -> A -> S>
```

Those angle brackets `<...>` contain *dependent types* — types that mention runtime values. We'll get to what that means later. For now, the point is: this signature is unreadable.

6.2 5.2 The Solution: Name It

```
functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
```

Now the signature becomes:

```
spec compose-xf : [Xf A B] -> [Xf B C] -> [Xf A C]
  :doc "Compose two transducers"
```

`functor` is Prologos's way of saying: "I'm naming a complex type pattern so I never have to write out the full thing." It's like TypeScript's `type` alias on steroids:

```
// TypeScript equivalent (roughly)
type Xf<A, B> = <S>(step: (s: S, b: B) => S) => (s: S, a: A) => S;
```

6.3 5.3 Progressive Metadata on `functor`

Just like `spec`, `functor` supports progressive metadata:

```
functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
  :compose xf-compose     ;; "this type composes via xf-compose"
  :identity id-xf         ;; "the identity element is id-xf"
  :laws (transducer-laws)  ;; "it must satisfy these algebraic laws"
```

You don't need `:compose`, `:identity`, or `:laws` to use `functor`. Start with just `:unfolds`. Add algebraic structure when you need it.

6.4 5.4 The Key Insight

The dependent type (`<...>`) lives in `:unfolds`, consulted only when the compiler needs it. The programmer-facing surface is just `Xf A B`.

Dependent types are the implementation substrate. The surface language speaks in domain terms.

This is Prologos's design philosophy in a nutshell. The deep theory is there when you need it — but it's behind a door you never have to open unless you want to.

7 Spiral 6: Channels and Protocols (Session Types)

7.1 6.1 The Problem: Communication Goes Wrong

If you've built distributed services, you know the pain: the client sends a JSON object, the server expects a different shape, and things silently break. Or worse: the client sends two messages when the server expects one, and the connection hangs.

These are *protocol bugs*. TypeScript can't catch them — the type system doesn't model sequences of messages. REST API schemas (OpenAPI) are documentation, not enforcement. gRPC is closer, but still doesn't verify that your client and server agree on the *order* of messages.

7.2 6.2 Session Types: Protocols as Types

Prologos can express communication protocols *in the type system*:

```
;; A session defines the shape of a conversation
session Greeter
  send name : String       ;; client sends a string
  recv greeting : String   ;; client receives a string
  end                      ;; conversation is over
```

This says: "A `Greeter` session starts with the client sending a `String` (the name), then the server responds with a `String` (the greeting), then the conversation ends."

The compiler checks *both sides*:

- The client must send a `String` first, then receive a `String`
- The server must receive a `String` first, then send a `String` (the *dual* — send becomes receive, receive becomes send)

If either side violates the protocol, it's a *compile-time error*.

7.3 6.3 Branching Protocols: Choice and Offer

Real protocols have branches. Think of an HTTP-like interaction:

```
session DatabaseService
  rec Loop                  ;; recursive protocol (can repeat)
    offer                   ;; server offers these choices:
      | :query              ;; client can select :query
          recv q : Query    ;;   send a query
          send r : Result   ;;   get a result back
          Loop              ;;   and go again
      | :close              ;; or client can select :close
          end               ;;   and we're done
```

The server =offer=s branches; the client =select=s one. This is like a tagged union of possible message sequences. The compiler verifies that the server handles every branch the client might choose.

7.4 6.4 Dependent Sessions: Protocols That Depend on Values

Here's where Prologos goes beyond what gRPC or OpenAPI can express:

```
session VecProtocol (A : Type)
  recv n : Nat             ;; client sends a number
  send v : [Vec A n]       ;; server sends a vector of THAT length
  end
```

The type of the *second* message depends on the *value* of the first. If the client sends `3`, the server must send a vector of length 3. Not "some vector" — *exactly* 3 elements. The compiler proves this.

This is a **dependent session type** — the protocol shape depends on runtime values. No mainstream language can express this.

8 Spiral 7: Capabilities and the Eventual Send

8.1 7.1 A Quick Grounding: What Are Capabilities?

Object-capability security (ocap) is a model where authority comes from *possession of a reference*, not from ambient identity checks (like ACLs or ambient auth). The idea originates in Mark Miller's E language (2000s) and is being standardized for decentralized networks by the OCapN pre-standardization group, with implementations in Spritely Goblins (Guile Scheme) and the Endo project (JavaScript/Hardened JS).

The core principles:

- **No ambient authority**: you can't "import" a database handle from thin air. Someone must hand you a reference.
- **Attenuation**: you can give someone a *weaker* version of your authority (read-only view, rate-limited proxy, etc.)
- **Transfer**: you can pass a capability to someone else — and if it's unique, passing it means *you no longer have it.*
- **Composition**: capabilities are just references — they compose like any other value.

If you've built systems with OAuth tokens, API keys, or dependency injection, you've been *approximating* ocap informally. The difference is enforcement: in Prologos, the compiler verifies these properties.

8.2 7.2 Channels as Capabilities

In Prologos, a typed channel *is* a capability. Consider:

```
session ReadOnlyDB
  rec Loop
    offer
      | :query
          recv q : Query
          send r : Result
          Loop
      | :close
          end
```

If a function receives a channel of type `ReadOnlyDB`, it can *only* query and close. It cannot insert, delete, or modify data — not because of a runtime ACL check, but because those operations don't exist in the session type. The *type itself is the attenuation boundary.*

This maps directly to the ocap principle of attenuation: the `ReadOnlyDB` session type is a *powerless facet* of a full `DatabaseService` — same database, smaller authority surface.

8.3 7.3 Linear Types Prevent Capability Duplication

Prologos's linear types (QTT — Quantitative Type Theory) add another layer. A channel marked with multiplicity `:1` (linear) cannot be duplicated:

```
;; If db-channel is linear (:1), you cannot do this:
let chan1 = db-channel
let chan2 = db-channel   ;; TYPE ERROR: db-channel already consumed
```

This means: if I give you my database channel, *I no longer have it.* That's capability transfer — enforced by the compiler. No need for runtime wrappers or trust assumptions.

8.4 7.4 The Eventual Send

The *eventual send* is a fundamental concept from Mark Miller's E language (and now central to OCapN's CapTP — the Capability Transport Protocol). The idea: instead of blocking on a remote call, you fire off a message and get back a *promise.* The remote side processes it asynchronously. Multiple messages can be in-flight simultaneously.

In JavaScript, this concept is making its way in via the TC39 proposal-eventual-send, with syntax like `E(target).method(args)`.

Prologos provides this natively via `!!` (non-blocking send) and `recv!` (non-blocking receive):

```
defproc db-client [queries : List Query] : List QueryResult
  let channel = [connect DatabaseService]

  ;; Fire all queries without waiting (eventual sends)
  let promises = [map [fn [q]
    select :query channel
    send q channel
    recv! channel           ;; recv! = eventual receive, returns promise
  ] queries]

  ;; Do other work while queries execute...
  [log "queries submitted, waiting..."]

  ;; Collect all results when ready
  let results = [map await promises]

  select :close channel
  results
```

The `recv!` operation returns a promise. Multiple queries fly out concurrently. `await` collects the results. The session type *still guarantees* that every query gets exactly one response — the protocol is preserved even in the asynchronous case.

This is analogous to E's *promise pipelining*: rather than waiting for each round trip, you send all your messages and let the promises resolve in whatever order the network delivers. The key difference: Prologos's session type statically proves that the promise chain is well-formed — every send has a matching receive, every branch is handled.

8.5 7.5 How This Compares to OCapN

OCapN / E Concept	Prologos Equivalent	Enforcement
Object reference	Typed channel endpoint	Compile-time type
Capability attenuation	Session type (can only do what the type says)	Compile-time type
No ambient authority	Linear types prevent copying channels	Compile-time QTT
Eventual send (`E(x).m()`)	`!!` / `send!` (non-blocking send)	Compile-time type
Promise	`recv!` returns a promise; `await` resolves	Compile-time type
Promise pipelining	Multiple `recv!` in-flight; session type tracks	Compile-time type
Vat (isolated object graph)	Process with linear channel context	Compile-time QTT
CapTP (capability transfer)	Channel passing (linear: transfer of authority)	Compile-time QTT
Sealer/Unsealer pairs	Existential types (Sigma) — pack/unpack	Compile-time type

The key difference from existing ocap implementations: in Spritely's Goblins, capabilities are enforced at *runtime* by the Guile Scheme VM. In Endo/Hardened JS, they're enforced by the SES (Secure ECMAScript) sandbox. In Prologos, capabilities are enforced at *compile time* by the type checker. If your program type checks, protocol violations are *impossible*. The type system *is* the capability system.

> "Logos with session types gives you ocap where the capability protocol is enforced by the compiler, not just convention." — from project design discussion

9 Spiral 8: Dependent Types (The Deeper Layer)

9.1 8.1 You've Already Used Them

If you read the `VecProtocol` example above and understood it, you've already used dependent types. A dependent type is just a type that mentions a runtime value:

```
spec replicate : <(n : Nat) -> A -> [Vec A n]>
```

"Given a number `n`, return a vector of *exactly* `n` elements." The return type depends on the argument value. That's it. That's a dependent type.

The angle brackets `<...>` mark the dependent part. Everything outside angle brackets is regular ML-style typing. This visual separator means you can *see* where dependency happens.

9.2 8.2 The Full Progressive Complexity Ladder

Let's revisit the levels, now that you have the intuition:

Level	What You Write	What It Means	TypeScr
0	`defn add [x y] [+ x y]`	No types (inference)	TS with
1	`spec add : Int -> Int -> Int`	Concrete types	`functio`
2	`spec id : A -> A`	Generics	`functio`
3	`spec sort : ... :where (Ord A)`	Constrained generics	`<A exte`
4	`spec fmap : ... :where (Functor F)`	Higher-kinded generics	No TS e
5	`functor Xf {A B} :unfolds ...`	Named type abstractions	`type Xf`
6	`spec replicate : <(n : Nat) -> ...>`	Dependent types (Pi)	No TS e
7	`spec filter : ... <result * proof>`	Dependent pairs (Sigma/existential)	No TS e

Levels 0-3 are familiar territory for any TypeScript developer. Level 4 is where Prologos starts going beyond what mainstream languages offer. Levels 5-7 are opt-in power tools.

You never have to leave the level you're comfortable at. Code at Level 1 and code at Level 7 coexist in the same codebase, call each other, and are type-checked together.

9.3 8.3 ?? : The "What Goes Here?" Hole

When working with complex types, Prologos provides typed holes — write `??` where you don't know what to put, and the compiler tells you what type it expects:

```
defn zip-with
  | [f vnil _]             -> vnil
  | [f _ vnil]             -> ??           ;; What goes here?
  | [f (vcons x xs) (vcons y ys)]
      -> [vcons [f x y] [zip-with f xs ys]]
```

The compiler responds:

```
Hole ?? at line 3
Expected type: Vec B 0
In context:
  f  : A -> A -> B
  xs : Vec A (suc n)
  ys : Vec A 0
Hint: The only value of type Vec B 0 is vnil
```

This is *interactive type-driven development*. The types guide you toward the correct implementation. Start with holes, let the compiler fill in the blanks. It's like pair programming with the type checker.

10 Spiral 9: The Big Picture — `spec`, `functor`, `session`

10.1 9.1 The Keyword Symmetry

Prologos has a deliberate symmetry in its specification language:

Keyword	What It Describes	Analogy
`spec`	Function type + metadata	TypeScript function signature
`trait`	A method that types must implement	TypeScript/Java interface method
`bundle`	A group of traits (AND conjunction)	`extends A & B & C`
`property`	A group of testable invariants	QuickCheck property suite
`functor`	A named, reusable type pattern	TypeScript `type` alias (richer)
`session`	A communication protocol	gRPC service definition (richer)

10.2 9.2 They All Compose

The keywords aren't isolated features — they build on each other:

```
;; A property group
property transducer-laws {A B C : Type}
  :holds [eq? [compose-xf id-xf xf] xf]       ;; identity law
  :holds [eq? [compose-xf [compose-xf f g] h]
              [compose-xf f [compose-xf g h]]] ;; associativity

;; A type abstraction
functor Xf {A B : Type}
  :doc "A transducer"
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
  :compose xf-compose
  :identity id-xf
  :laws (transducer-laws)

;; A function spec using the above
spec map-xf : [A -> B] -> [Xf A B]
  :doc "Transducer that applies f to each element"
  :examples
    - [into-list [map-xf inc] '[1 2 3]] => '[2 3 4]

;; A trait using all of the above
trait Transducible {C : Type -> Type}
```

```
  :laws (transducer-fusion C)
  to-xf : {A B : Type} -> [A -> B] -> [Xf A B]
```

The `functor` names the type. The `property` states its laws. The `spec` uses both. The `trait` packages it for generic dispatch. Everything is readable. The dependent types are present exactly once, in `:unfolds`, consulted only when needed.

11 Where To Go From Here

11.1 The Progressive Complexity Promise

Prologos is designed so that:

1. A developer who uses Level 0-2 (no types or simple generics) is productive immediately
2. A developer who uses Level 3-4 (traits + higher-kinded types) writes highly reusable library code
3. A developer who uses Level 5-7 (functors + dependent types) writes code with compiler-verified correctness guarantees
4. A developer who uses session types writes distributed systems where protocol violations are compile-time errors
5. A developer who uses linear types writes capability-secure code where authority is tracked by the compiler

All five levels coexist. Code at any level can call code at any other level. The type checker handles the complexity; you choose how much specification you want to write.

> "Dependent types are the fabric upon which everything is built — and can be reached for in pure form if ever needed — but otherwise use `functor`, `spec`, etc., with their implicit maps."

12 Further Reading

12.1 Prologos Design Documents

- `docs/tracking/2026-02-22_EXTENDED_SPEC_DESIGN.md` — the full `spec=/=functor=/=property` design (the document that inspired this tutorial)

- `docs/tracking/principles/LANGUAGE_VISION.md` — the overall vision
- `docs/otto_conversation.org` — a conversation exploring session types, OCapN, and capabilities in depth

12.2 The Codebase

- `lib/prologos/core/` — trait definitions and standard library
- `lib/prologos/data/` — collection types (List, PVec, Map, Set)
- `tests/` — comprehensive tests showing every feature in action
- `docs/spec/grammar.ebnf` — the formal grammar of the language

12.3 Capability-Based Security

- OCapN Pre-standardization Group — the community defining decentralized capability networking
- CapTP Draft Specification — the wire protocol for capability transfer between machines
- Spritely Goblins OCapN Docs — reference implementation in Guile Scheme
- Endo / Hardened JavaScript — JavaScript-native ocap (SES, eventual send, Agoric)
- TC39 Proposal: Eventual Send — bringing E-style eventual send to JavaScript
- Awesome Object Capabilities — curated reading list on the ocap model
- Mark S. Miller, *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control* (2006) — the foundational dissertation on E and capabilities